

PROJECT DOCUMENTATION · CONFIDENTIAL

# Bullion Infrastructure

Active-active multi-cloud Kubernetes platform — running AWS EKS and GCP GKE as a single, mesh-joined cluster with cross-cloud load balancing, delivered entirely through infrastructure-as-code and CI/CD.

✓ Active-active cross-cloud mesh — LIVE & verified (traffic balances EKS ⇌ GKE)

Repository: Olisaemeka111/bullion-infrastructure · Version 1.0 · 2026-06-23

AWS/EKS

GCP/GKE

Mesh

CI/CD

Security

Observability

Data

Roadmap

# 0 Document control & contents

| FIELD           | VALUE                                                                               |
|-----------------|-------------------------------------------------------------------------------------|
| Title           | Bullion Infrastructure — Project Documentation                                      |
| Version / date  | 1.0 · 2026-06-23                                                                    |
| Status          | <b>Operational</b> — active-active mesh live; data layer + control-plane on roadmap |
| Audience        | Engineering leadership, platform/SRE teams, security, auditors                      |
| Owner           | Platform Engineering                                                                |
| Source of truth | Git repo (Terraform, workflows, mesh manifests, this document)                      |

## Contents

|    |                                         |      |
|----|-----------------------------------------|------|
| 1  | Executive summary                       | \$1  |
| 2  | Business context & objectives           | \$2  |
| 3  | Solution architecture                   | \$3  |
| 4  | Infrastructure components               | \$4  |
| 5  | Service mesh (Istio multi-primary)      | \$5  |
| 6  | CI/CD & automation                      | \$6  |
| 7  | Security & compliance                   | \$7  |
| 8  | Observability                           | \$8  |
| 9  | Data layer (CockroachDB — planned)      | \$9  |
| 10 | Reference control plane (offline model) | \$10 |
| 11 | Operations runbook                      | \$11 |
| 12 | Testing & verification                  | \$12 |
| 13 | Implementation journey                  | \$13 |
| 14 | Current state & roadmap                 | \$14 |
| 15 | Cost                                    | \$15 |
| 16 | Risks & recommendations                 | \$16 |
| 17 | Appendices                              | \$17 |

## 1 Executive summary

**What this is.** Bullion Infrastructure is a production-style platform that runs Kubernetes in two public clouds — Amazon EKS and Google GKE — and makes them behave as **one active-active cluster**. A single application, deployed to both clouds, is served as one global service with traffic load-balanced across both providers simultaneously; if one cloud degrades, traffic automatically shifts to the other.

**Why it matters.** It removes single-cloud dependency: the platform survives a whole-cloud outage, avoids vendor lock-in, and lets workloads run close to users in multiple regions — without operators managing failover by hand.

2

CLOUDS, ACTIVE-ACTIVE

4

WORKER NODES (2/CLOUD)

100%

IAC + CI/CD DEPLOYED

0

STATIC CLOUD KEYS (OIDC)

**Outcome (verified).** The clusters are joined into one Istio mesh and load balancing across clouds is confirmed: 20 requests from the AWS cluster were served **12 by AWS and 8 by GCP**. The whole platform is reproducible from Git and deployed through gated GitHub Actions pipelines with keyless cloud authentication.

### Delivered

- Two managed Kubernetes clusters (EKS + GKE), provisioned by Terraform.
- An Istio **multi-primary** service mesh joining both clusters across clouds (mTLS).
- **Active-active load balancing** of a global service across both clouds, with automatic failover (outlier ejection).
- A **cross-cloud private network** (AWS↔GCP HA VPN + BGP, 4 tunnels) — live.
- A **central distributed database** (CockroachDB multi-region, 4 nodes across both clouds over the VPN), deployed via CI/CD.
- A real **global active-active application** (the board game) built from source to GHCR and served from both clouds behind public ingress LoadBalancers.
- Full **CI/CD** (GitHub Actions) with keyless OIDC auth and approval gates.
- A **DevSecOps security gate** on every push/PR — SAST (CodeQL), secret scanning (Gitleaks), dependency/vuln scanning (Trivy, OWASP Dependency-Check) and IaC scanning (Checkov, tfsec, KICS), plus Ruff linting.
- Centralised **observability** (Prometheus/Grafana).

### On the roadmap

- In-cluster control-plane/dashboard (the agentic reconciler running on-cluster).
- A single global DNS endpoint (one hostname) fronting both ingresses — optional.
- Production hardening: CockroachDB TLS, least-privilege IAM, private GKE endpoint.

**Commercial note.** The environment currently runs on a GCP free trial and a pay-as-you-go AWS account for validation. Production hardening (least-privilege IAM, private API endpoints, quota uplifts, the data layer) is scoped in \$14/\$16.

## 2 Business context & objectives

### Objectives

- **Resilience:** survive a full cloud-provider outage.
- **No lock-in:** one platform, portable across clouds.
- **Active-active:** use all capacity, not idle standby.
- **Automation:** everything reproducible via IaC + CI/CD.
- **Security by default:** keyless auth, mTLS, gated changes.

### Non-goals (this phase)

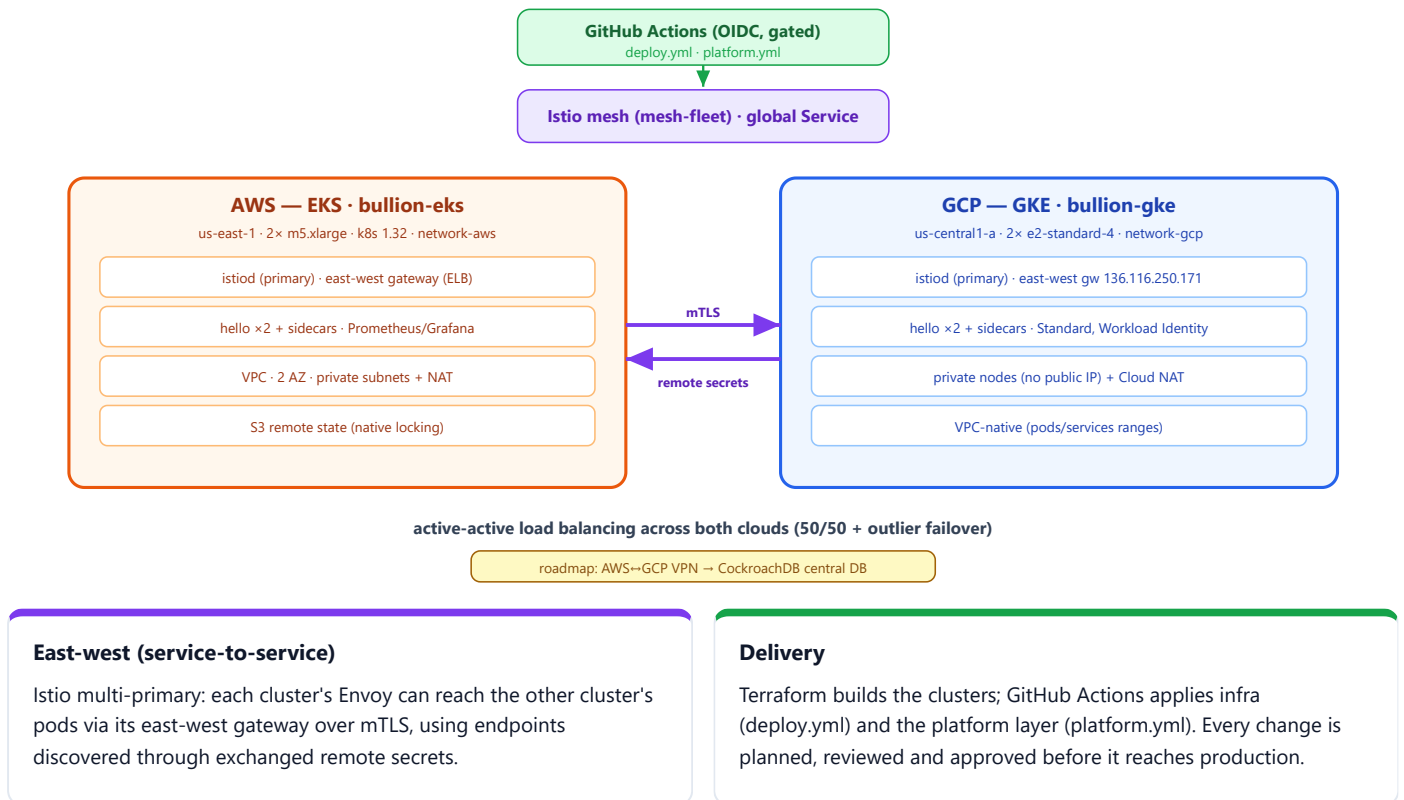
- Production-grade least-privilege IAM (currently broad for validation).
- Stateful workloads at scale (data layer is on the roadmap).
- Azure (no subscription available; cleanly disabled, re-enableable).

### Design principles

| PRINCIPLE                      | HOW IT'S REALISED                                                                                                                                 |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Two control planes, one fabric | Each cluster keeps its own K8s + istiod; the mesh + global LB make them one logical platform (you never stretch one control plane across clouds). |
| Declarative & reproducible     | Terraform for infra; manifests for the mesh; GitHub Actions to apply — all in Git.                                                                |

|                 |                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------|
| Keyless & gated | GitHub OIDC to each cloud; production environment requires human approval.                      |
| Active-active   | Global service spans both clusters; DestinationRule splits 50/50; outlier detection fails over. |

### 3 Solution architecture



**AWS EKS — bullion-eks**

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| Account / region | 730767193290 / us-east-1                                                      |
| Kubernetes       | 1.32 (managed control plane)                                                  |
| Nodes            | 2 × m5.xlarge (4 vCPU/16GB), on-demand                                        |
| Networking       | VPC, 2 AZ, private node subnets + single NAT, public subnets, EKS subnet tags |
| Access           | EKS access entries (CI role + operator = cluster-admin)                       |
| Mesh ports       | node SG opened for istiod webhook 15017 + xDS 15012                           |

**GCP GKE — bullion-gke**

|                |                                                             |
|----------------|-------------------------------------------------------------|
| Project / zone | project-f38bfcd6-70ff-4a8d-9cd / us-central1-a              |
| Mode           | GKE Standard (zonal), Workload Identity enabled             |
| Nodes          | 2 × e2-standard-4 (4 vCPU/16GB)                             |
| Networking     | VPC-native; private nodes (no public IP) + Cloud NAT egress |
| Control plane  | public endpoint (open) for CI/operator access               |
| Constraint     | private nodes chosen to fit free-trial IP quota             |

## Shared / cross-cutting

| COMPONENT    | DETAIL                                                                                                              |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| IaC          | Terraform ≥ 1.10, modules: <code>eks</code> , <code>gke</code> , <code>global_dns</code> , <code>cross_cloud</code> |
| Remote state | S3 bucket <code>fleet-mini-tfstate-59998ac4</code> , native state locking (no DynamoDB)                             |
| Naming       | <code>bullion-*</code> prefix across clusters + networks                                                            |
| Azure (AKS)  | module present but commented out; <code>enable_azure=false</code> (no subscription)                                 |

**5 Service mesh — Istio multi-primary**

Both clusters form one mesh ( `meshID = mesh-fleet` ). Multi-primary = an independent istiod per cluster; multi-network = traffic crosses clouds via east-west gateways.

| ELEMENT                 | IMPLEMENTATION                                                                                                                                        |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Trust                   | Shared root CA issues each cluster's istiod intermediate (the <code>cacerts</code> secret) → cross-cloud mTLS works.                                  |
| Control plane           | One istiod per cluster (no single point of failure).                                                                                                  |
| Cross-cluster transport | East-west gateways (LoadBalancer): EKS ELB hostname; GKE IP 136.116.250.171.                                                                          |
| Discovery               | Remote secrets ( <code>istio-remote-secret-eks/-gke</code> ) let each istiod read the other's endpoints.                                              |
| Network identity        | <code>topology.istio.io/network</code> on istio-system (network-aws / network-gcp) — required for cross-network routing.                              |
| Global service          | <code>hello.demo.svc.cluster.local</code> deployed to both; <code>DestinationRule</code> 50/50 cross-region + <code>outlierDetection</code> failover. |

**Verification.** 20 calls from an EKS client pod → **12 AWS / 8 GCP**. Cross-cloud active-active load balancing confirmed.

## 6 CI/CD & automation

| PIPELINE                                          | RESPONSIBILITY                                                                         | TRIGGER / GATE                                   |
|---------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------|
| <code>ci.yml</code>                               | Ruff lint + 80 unit/integration tests (stdlib) + <code>terraform fmt / validate</code> | every push/PR · no cloud creds                   |
| <code>security.yml</code>                         | DevSecOps scans: CodeQL, Gitleaks, Trivy, OWASP Dependency-Check, Checkov, tfsec, KICS | every push/PR · gate on CRITICAL/HIGH            |
| <code>deploy.yml</code>                           | Terraform infra (EKS, GKE, VPN): plan → apply → destroy                                | PR=plan · merge=apply (gated) · dispatch=destroy |
| <code>platform.yml</code>                         | Istio join + global app + observability across both clusters                           | dispatch / mesh changes (gated)                  |
| <code>database.yml</code> · <code>game.yml</code> | CockroachDB (both clusters) · board game (build → GHCR → both clusters)                | dispatch / path changes                          |

### Flow

Open PR → lint + tests + security scans run → automated **plan** posted as a comment → review → merge → **apply** pauses for approval in the `production` environment → resources provision.

### Keyless auth

AWS via IAM role + GitHub OIDC; GCP via Workload Identity Federation; tokens are minted per-run and short-lived. No static cloud keys live in GitHub.

### Shift-left security gate ( `security.yml` )

Every push/PR runs, with no cloud credentials, gating on **CRITICAL/HIGH** and any secret: **SAST** = CodeQL (Python) · **Secrets** = Gitleaks (full history) · **Vulns/deps** = Trivy + OWASP Dependency-Check (CVSS ≥ 7) · **laC** = Checkov + tfsec + KICS over `iac/terraform`. Findings upload to the Security tab as SARIF. CodeQL and Gitleaks pass clean; the IaC and dependency scanners surface real Terraform hardening items for triage.

## 7 Security & compliance

### Identity & access

- GitHub OIDC → AWS IAM role / GCP WIF (no static keys).
- EKS access entries map identities to cluster RBAC.
- GKE Workload Identity for pod-level GCP access.
- Production deploys require human approval (GitHub environment).

### In-transit & supply chain

- Mesh-wide **mTLS** (shared CA) for service-to-service traffic.
- Reference admission model: signed images from trusted registries, PSS.
- Remote state encrypted in S3 with locking.

**Known hardening items (validation environment).** The bootstrap AWS key was shared during setup and must be **rotated**; the operator IAM user currently holds `AdministratorAccess` (scope down); the GKE control-plane endpoint is publicly reachable (restrict via authorized networks); and CockroachDB currently runs in insecure mode (enable TLS for production). Deploy identifiers are held as GitHub Actions secrets; for runtime app secrets a cloud-native secret manager + External Secrets is recommended.

## 8 Observability

### Deployed

`kube-prometheus-stack` (Prometheus + Grafana + node-exporter) on EKS; GKE uses Google Managed Service for Prometheus. Istio contributes per-service golden signals (latency, errors, throughput, saturation).

### Fleet view (design)

Per-cluster Prometheus federates into a single fleet-wide view; the reference control plane also exposes Prometheus metrics + an offline dashboard (\$10).

## 9 Data layer — CockroachDB (deployed)

Active-active *traffic* needs active-active *data*. A distributed multi-master database lets every cloud read and write and survive a region loss.

| ASPECT       | IMPLEMENTATION                                                                                                          |
|--------------|-------------------------------------------------------------------------------------------------------------------------|
| Engine       | CockroachDB <code>v24.3.5</code> as a <b>DaemonSet</b> (one node per K8s node) joined into one logical DB               |
| Networking   | <b>hostNetwork</b> — each node advertises its routable <b>node IP</b> (pod IPs aren't routable cross-cloud)             |
| Connectivity | node-to-node (:26257) over the AWS→GCP <b>HA VPN</b> (cross_cloud module); <code>--join</code> built from live node IPs |
| Deployment   | CI/CD via <code>database.yml</code> (discovers node IPs → applies to both clusters → <code>init</code> once)            |
| Status       | <b>LIVE — 4 nodes across both clouds (2 EKS + 2 GKE)</b>                                                                |

**Hardening.** The cross-cloud VPN is up and the cluster is live in **insecure** mode for this validation; enabling TLS (`cert-manager / cockroach cert`) is the production-hardening step.

## 10 Reference control plane (offline model)

The repository also contains an agent-driven control-plane **reference implementation** (Python) that models the full lifecycle — reconciliation loop, provider abstraction, node agent, security gate, networking fabric, observability — and runs entirely offline with **80 passing tests**. It proves the design (including the active-active traffic model in `networking/global_lb.py` + `sim/active_active.py`) deterministically.

On the live managed clusters, its responsibilities (provision / scale / self-heal) are met by Terraform + managed node groups; the **Provider** interface is the documented seam for driving real clouds directly in a future phase. An in-cluster deployment of this control plane + dashboard is on the roadmap.



## 11 Operations runbook

### Access

```
aws eks update-kubeconfig --name bullion-eks --region us-east-1 --alias eks
gcloud container clusters get-credentials bullion-gke --zone us-central1-a --project project-f38bfd6-70ff-4a8d-9cd
```

### Deploy / change

Open a PR (auto **plan**) → review → merge → approve the gated **apply**. Platform layer: Actions → **platform** → Run workflow → approve.

### Verify active-active

```
kubectll --context eks -n demo run curl --image=curlimages/curl --restart=Never --command -- sleep 3600
kubectll --context eks -n demo exec curl -c curl -- sh -c \
  "for i in 1 2 3 4 5 6 7 8; do curl -s hello.demo.svc.cluster.local; echo; done"
# expect a mix of "hello from aws" and "hello from gcp"
```

### Failure handling

A cloud/endpoint failure is ejected by **outlierDetection** and traffic shifts to the survivor automatically. No manual failover is required for the traffic layer.

### Teardown (stop costs)

Actions → deploy → Run workflow → action=destroy → approve

## 12 Testing & verification

| LAYER                      | COVERAGE                                                                                                |
|----------------------------|---------------------------------------------------------------------------------------------------------|
| Unit/integration (offline) | 80 tests (smoke/unit/integration/chaos) — control logic, security gate, networking, active-active model |
| IaC                        | <b>terraform fmt</b> + <b>validate</b> in CI on every change                                            |
| Live mesh                  | cross-cloud LB verified (12 AWS / 8 GCP from an EKS client)                                             |

## 13 Implementation journey — blockers resolved

| SYMPTOM                                                  | RESOLUTION                               |
|----------------------------------------------------------|------------------------------------------|
| EKS rejected Kubernetes 1.29                             | upgraded to 1.32                         |
| GKE Autopilot can't host self-managed Istio / DaemonSets | switched GKE to Standard                 |
| Node pool rejected GKE_METADATA                          | enabled Workload Identity on the cluster |
| istiod wouldn't schedule on medium nodes                 | moved to 4 vCPU/16 GB nodes              |
| GCP free-trial IP quota (4) exceeded                     | private nodes + Cloud NAT                |
| Terraform state drift from partial applies               | fresh plan reconciled                    |
| EKS sidecar-injector webhook timeout                     | opened node SG :15017 / :15012           |
| DestinationRule distribute+failover rejected             | distribute + outlierDetection            |
| Cross-cluster traffic stayed local                       | added topology.istio.io/network label    |

## 14 Current state & roadmap

| CAPABILITY                                                                   | STATE                   |
|------------------------------------------------------------------------------|-------------------------|
| EKS + GKE provisioned (IaC + CI/CD)                                          | ✓ live                  |
| Istio multi-primary mesh — clusters joined                                   | ✓ live                  |
| Active-active cross-cloud load balancing                                     | ✓ verified              |
| North-south ingress gateways (public LB :80)                                 | ✓ live                  |
| Board game — global active-active app (GHCR → both clouds)                   | ✓ live                  |
| Observability (Prometheus/Grafana)                                           | ✓ live                  |
| DevSecOps scans + lint (CodeQL/Gitleaks/Trivy/OWASP/Checkov/tfsec/KICS/Ruff) | ✓ wired                 |
| Cross-cloud VPN (AWS↔GCP)                                                    | ✓ live (4 tunnels)      |
| Central CockroachDB (multi-region, via CI/CD)                                | ✓ live — 4 nodes        |
| In-cluster control plane + dashboard                                         | planned                 |
| Global DNS (single-endpoint north-south LB)                                  | module built (optional) |

## 15 Cost

| ITEM                                     | INDICATIVE COST          |
|------------------------------------------|--------------------------|
| EKS control plane                        | ~\$0.10 / hr             |
| EKS nodes (2× m5.xlarge)                 | ~\$0.38 / hr             |
| GKE (Standard) + nodes                   | on GCP free-trial credit |
| LoadBalancers / NAT / (VPN when enabled) | hourly + data egress     |

Run the destroy pipeline when idle. Cross-cloud data egress is the main variable cost once the VPN + database are enabled.

## 16 Risks & recommendations

| RISK                                             | RECOMMENDATION                                                          |
|--------------------------------------------------|-------------------------------------------------------------------------|
| Broad IAM (Administrator) + shared bootstrap key | rotate the key; scope roles to least privilege                          |
| Public GKE control-plane endpoint                | restrict with master authorized networks                                |
| Free-trial quotas (GCP IPs/CPU)                  | request uplifts before scaling / production                             |
| CockroachDB runs in insecure mode                | enable TLS (cert-manager / <code>cockroach cert</code> ) for production |
| Runtime app-secret management                    | adopt a cloud-native secret manager + External Secrets Operator         |

## 17 Appendices

### A · Repository layout

```
iac/terraform/ EKS, GKE, global_dns, cross_cloud + S3
backend
mesh/install/ Istio multi-primary (CA, IOPs, east-west +
ingress, secrets)
mesh/apps/ global service + active-active
DestinationRule + board game
mesh/observability/ node-exporter + federated Prometheus
```

### B · Key identifiers

|             |                                |
|-------------|--------------------------------|
| AWS account | 730767193290                   |
| EKS cluster | bullion-eks (us-east-1)        |
| GCP project | project-f38bfcd6-70ff-4a8d-9cd |
| GKE cluster | bullion-gke (us-central1-a)    |

```
mesh/data/      CockroachDB multi-region (cross-cloud, via
database.yml)
.github/workflows/ ci · security · deploy · platform ·
database · game
ruff.toml      Python lint config (Ruff)
control_plane/ providers/ agent/ security/
networking/ workflows/ sim/ tests/  (offline reference)
DEPLOY · CICD · INFRASTRUCTURE · ARCHITECTURE · RUNBOOK ·
TESTING
```

|              |                                      |
|--------------|--------------------------------------|
| Repo         | Olisaemeka111/bullion-infrastructure |
| State bucket | fleet-mini-tfstate-59998ac4          |

## C · Glossary

### Multi-primary

An istiod control plane in each cluster.

### Multi-network

Clusters on different networks joined via east-west gateways.

### Active-active

All clouds serve traffic at once (vs cold standby).

### OIDC

Keyless federated auth — short-lived tokens, no stored secrets.